

Important ML Kernels for GPU Programming Interviews

This guide covers the mathematical purpose, CUDA execution model, numerical issues, and optimization strategy of the ML kernels most often discussed in GPU, systems, and ML-infrastructure interviews.

Pseudocode is CUDA-like; actual production choices depend on GPU generation, datatype, tensor shape, and framework.

Topics

- [Softmax](#)
- [LayerNorm](#)
- [RMSNorm](#)
- [ReLU and GELU](#)
- [Cross Entropy](#)
- [Matrix Multiplication](#)
- [Attention](#)
- [FlashAttention-Style Tiled Attention](#)
- [Quantization Kernels](#)
- [Final Cross-Kernel Interview Map](#)

ReLU and GELU

1. Overview

Definition. ReLU and GELU are elementwise activation functions that introduce nonlinearity into neural networks.

```
ReLU(x) = max(0, x)
GELU(x) = x * Phi(x)
          ≈ 0.5*x*(1 + tanh(sqrt(2/pi)*(x + 0.044715*x^3)))
```

Phi is the standard normal cumulative distribution function. ReLU is cheap and common in CNNs and MLPs; GELU is smooth and common in Transformers. Interviewers ask about them because they test basic neural-network knowledge, elementwise GPU mapping, branching, approximations, vectorization, fusion, and backward computation.

2. Core Idea

ReLU is a hard gate: positive signals pass unchanged and negative signals are blocked. GELU is a soft probabilistic gate: strongly positive inputs mostly pass, strongly negative inputs are mostly suppressed, and values near zero transition smoothly.

x	-2	-0.5	0	0.5	2
ReLU	0	0	0	0.5	2
GELU	~ -0.046	~ -0.154	0	~ 0.346	~ 1.954

On a GPU these are embarrassingly parallel: thread `i` loads `x[i]`, evaluates the function, and stores `y[i]`. The challenge is not synchronization but moving memory efficiently, choosing accurate/fast math, handling vector tails, and fusing the activation into a producer such as GEMM+bias.

3. Important Subtopics

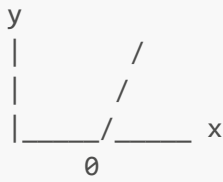
Subtopic	What it means and why it matters	Example	Common interview angle
Elementwise parallelism	Outputs do not depend on neighboring elements.	One thread handles one or several contiguous values.	No shared memory or reduction required.
ReLU branch	Can be implemented as <code>max(x, 0)</code> or predication.	Avoid divergent control flow.	Is <code>if</code> necessarily slow? Compiler often predicates it.
GELU exact/approximate	Exact form uses <code>erf</code> ; common approximation uses <code>tanh</code> .	Framework flag may select version.	Accuracy and checkpoint compatibility.
Derivatives	ReLU derivative is piecewise; GELU derivative is smooth.	Training backward kernels.	Behavior at zero.
Dead ReLU	A unit can remain in the negative region with zero gradient.	Motivates Leaky ReLU variants.	Optimization effect, not a GPU race.
Fusion	Activation commonly appears in GEMM epilogues with bias.	<code>GELU(XW+b)</code> .	Avoid intermediate tensor traffic.
Vectorization	Load/store packed values when aligned.	<code>half2</code> , <code>float4</code> .	Tail and alignment correctness.
In-place operation	Safe when each output depends only on its corresponding input and backward needs are handled.	ReLU can save a mask/output.	Autograd and aliasing constraints.

4. Real-World Example

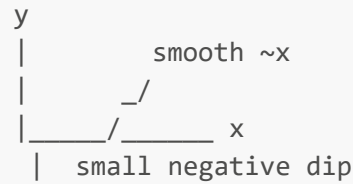
The Transformer MLP computes a projection, adds bias, applies GELU, and projects again. A library can implement the first GEMM's epilogue as bias+GELU so its accumulators are converted directly into activated output, avoiding a separate global read/write and kernel launch.

5. Diagrams / Mental Models

ReLU:



GELU:



Kernel property	ReLU	GELU
Cross-thread dependency	None	None
Expensive math	No	erf or tanh approximation
Typical bottleneck	Memory bandwidth/launch	Memory bandwidth or special-function throughput
Natural fusion point	Convolution/GEMM epilogue	GEMM+bias epilogue

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. Why are activations needed?	Stacking only linear layers remains one linear transformation; activations add nonlinearity.	Representational capacity.	Saying they merely normalize values.
2. ReLU versus GELU?	ReLU is a hard zero/identity gate; GELU is a smooth input-dependent gate with nonzero negative outputs.	Cost, smoothness, typical usage.	Saying GELU equals sigmoid.
3. Why is ReLU GPU-friendly?	Every element is independent and requires only a comparison/select.	Massive parallelism, coalesced memory.	Claiming shared memory is essential.
4. Does an if always cause warp divergence?	Only if lanes take different control paths and the compiler emits branches; simple ReLU is often compiled to predication/max.	Inspect generated code, don't assume.	Treating source syntax as hardware behavior.
5. Exact versus approximate GELU?	Exact GELU uses erf ; tanh approximation is close and can be faster, but forward/backward must follow the selected definition.	Numerical/compatibility tradeoff.	Mixing exact forward with approximate backward.

Question	Clear answer	Expected key points	Common mistake
6. What is ReLU's derivative at zero?	It is mathematically undefined; frameworks choose a convention, commonly zero.	Subgradient convention.	Claiming a unique mathematical derivative.
7. Why fuse bias and activation?	It avoids materializing the pre-activation and another kernel launch.	Memory traffic and latency.	Focusing only on arithmetic reduction.
8. Can activation run in place?	Often yes, but training may need the input or enough information for backward, and aliasing rules must permit it.	Autograd/storage tradeoff.	Declaring it always safe.
9. How is the grid sized?	Cover N elements with ceiling division; each thread may process a vector or grid-stride loop.	Bounds check and coalescing.	Ignoring tail elements.
10. What determines GELU performance?	Bytes moved, math implementation, datatype conversion, launch overhead, and fusion opportunities.	Shape-specific profiling.	Assuming the tanh instruction alone determines runtime.

7. Deep-Dive Questions

1. **Derive exact GELU's derivative.** For $y=x*\Phi(x)$, $dy/dx=\Phi(x)+x*\phi(x)$, where $\phi(x)=\exp(-x^2/2)/\sqrt{2*\pi}$.
2. **What should backward save?** ReLU may use input sign or output; GELU usually needs the input or a recomputable approximation. Saving costs bandwidth/memory, while recomputation costs math.
3. **How does fusion change precision?** A fused epilogue may apply activation to FP32 accumulators before casting, whereas separate kernels may round to FP16 first; results can legitimately differ.
4. **When is a standalone activation kernel inefficient?** For small tensors launch overhead dominates; for large tensors it writes and rereads an intermediate that a producer/consumer fusion could eliminate.
5. **How would you choose vector width?** Respect pointer alignment, contiguous length, datatype, and register pressure; provide scalar/predicated tail handling and benchmark representative shapes.

8. Comparison Tables

Property	ReLU	GELU	Leaky ReLU
Negative side	Zero	Smooth small values	alpha*x
Smooth at zero	No	Yes	Usually no
Compute cost	Very low	Higher	Very low
Zero gradient for negative x	Yes	Not generally	No
Common use	CNNs/general MLPs	Transformers	Avoiding dead units

GELU form	Expression	Tradeoff
Exact	$0.5 * x * (1 + \text{erf}(x / \sqrt{2}))$	Definition-faithful, special-function cost
Tanh approximation	$0.5 * x * (1 + \tanh(c * (x + 0.044715x^3)))$	Widely used, close approximation

9. Common Mistakes

- Calling GELU a zero-clipping activation.
- Using a separate kernel when the library GEMM epilogue already supports the activation.
- Assuming source-level `if` guarantees costly divergence.
- Ignoring the chosen GELU approximation in backward/reference tests.
- Forgetting tail handling with packed loads.
- Overlooking that low arithmetic intensity makes memory traffic central.

10. Edge Cases / Special Cases

- ReLU at exactly zero uses a framework-defined subgradient.
- Signed zero, NaN, and infinity behavior can differ by `max` intrinsic and fast-math mode.
- GELU approximations diverge slightly for extreme inputs.
- In-place activation may violate aliasing/autograd rules.
- Misaligned tensors cannot safely use assumed-aligned vector loads.

11. How to Explain in Interview

"ReLU is a hard elementwise gate and GELU is a smooth probabilistic gate used heavily in Transformers. Both map trivially across GPU threads; ReLU is mostly a memory operation, while GELU adds special-function math. The important optimization is usually vectorized coalesced access and fusion into the preceding GEMM or bias epilogue."

12. Quick Revision Notes

- ReLU: `max(0, x)`; GELU: `x * Phi(x)`.
- Independent elements mean no synchronization.
- ReLU backward needs a sign mask; GELU backward needs input/recomputation.
- Fusion saves an intermediate read/write.
- Trap: exact and approximate GELU must be distinguished.

13. Practice Tasks

1. Implement scalar CPU ReLU and exact/approximate GELU.
2. Write a vectorized CUDA activation kernel with safe tails.
3. Compare branch, ternary, and `max` generated code.
4. Implement backward and check with finite differences away from ReLU zero.
5. Fuse bias+activation and profile it against separate kernels.

14. Final Cheat Sheet

Item	Must remember
Core definition	Elementwise nonlinear gates; hard for ReLU, smooth for GELU.
Why it matters	Gives neural networks nonlinear capacity.
Most asked	Derivatives, divergence, approximation, fusion, bandwidth.
Main comparison	ReLU is cheaper; GELU is smooth and common in Transformers.
One-line answer	"Map one lane to contiguous values and fuse the activation with its producer when possible."

Cross Entropy

1. Overview

Definition. Cross entropy measures how poorly a predicted probability distribution matches a target distribution:

$$H(q, p) = -\sum_c q_c \log(p_c)$$

For a one-hot class target t , this becomes $-\log(p_t)$. In classification, implementations normally consume logits and compute log-softmax plus negative log-likelihood in one stable operation. It is used to train image classifiers, language models, recommendation models, and token predictors. Interviewers ask because it connects probability, loss gradients, stable log-sum-exp, reductions, label formats, and fused kernels.

2. Core Idea

Cross entropy penalizes the model for assigning low probability to the correct outcome. If the correct class receives probability 0.9 , the loss is $-\log(0.9) \approx 0.105$; at 0.01 , it is ≈ 4.605 .

Never compute softmax, store probabilities, then take a log if logits are available. For target t :

```
m      = max_c z_c
logsum  = m + log(sum_c exp(z_c - m))
loss    = logsum - z_t
```

This fused log-sum-exp form avoids $\log(0)$, prevents exponent overflow, and avoids materializing a probability matrix. A GPU kernel reduces across classes for each sample/token, writes one loss (or accumulates a requested reduction), and backward produces $\text{softmax}(z) - \text{target}$.

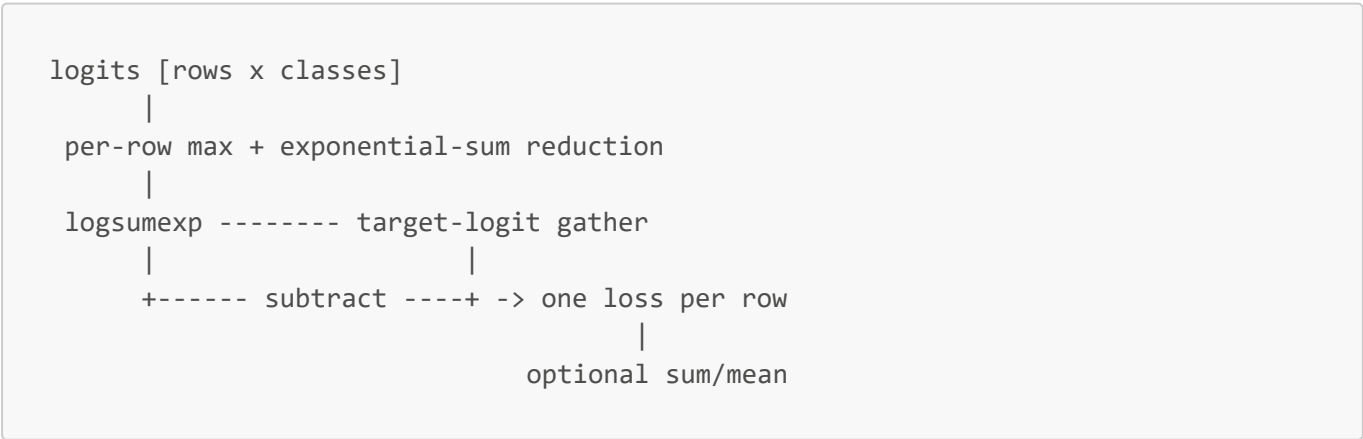
3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Log-sum-exp	Stable normalization in log space.	$m + \log(\sum \exp(z - m))$.	Derive from fused softmax+NLL.
Sparse labels	Target is an integer class index.	Language-model token ID.	Read only the target logit for NLL term.
Dense/soft labels	Target is a distribution.	Distillation or label smoothing.	Need weighted sum over all classes.
Reduction mode	Return per-item loss, sum, or mean.	Mean may exclude ignored targets.	Correct denominator.
Ignore index	Certain targets contribute neither loss nor gradient.	Padding tokens.	Do not include them in mean count.
Class weights	Scale contribution by target class.	Imbalanced classification.	Apply consistently in forward/backward.
Label smoothing	Mix one-hot target with a broad distribution.	Prevent overconfidence.	Changes loss and gradient for all classes.
Backward	Gradient w.r.t. logits is scaled $p - q$.	Simple after normalization.	Reduction scaling and weights.

4. Real-World Example

An LLM produces logits shaped `[tokens, vocabulary]`, sometimes tens of thousands of classes per row. Fused cross entropy streams each vocabulary row, computes stable log-sum-exp, gathers the correct token logit, and emits one loss. It avoids storing a huge softmax tensor when only the scalar loss and logits gradient are needed.

5. Diagrams / Mental Models



Target probability for correct class	Loss
0.99	0.010
0.50	0.693

Target probability for correct class	Loss
0.10	2.303
0.01	4.605

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What does cross entropy measure?	Expected negative log probability under the target distribution.	$-\sum q \log p.$	Calling it classification accuracy.
2. Why accept logits?	A fused log-softmax calculation is more stable and avoids storing probabilities.	Log-sum-exp.	Taking $\log(\text{softmax})$ separately.
3. What is the one-hot loss?	$\text{logsumexp}(\text{logits}) - \text{logit}[\text{target}]$.	Target gather plus row reduction.	Computing $-\log(\text{logit}[\text{target}])$.
4. What is the gradient?	For logits, $p-q$, adjusted by weights and output-reduction scale.	Softmax-cross-entropy simplification.	Giving derivative with respect to probabilities instead of logits.
5. Why subtract the maximum?	It stabilizes exponentiation without changing normalized probabilities/log-sum-exp.	Same principle as stable softmax.	Omitting the max add-back in log-sum-exp.
6. How does label smoothing change targets?	It distributes some target mass to other classes, so all classes receive nonzero target terms.	Define exact smoothing convention.	Assuming only the correct-class term changes.
7. How should ignored labels affect a mean?	They contribute zero and are excluded from the denominator.	Valid-item count.	Dividing by total rows including padding.
8. Sparse versus dense targets?	Sparse targets store one class index; dense targets store a distribution and require an all-class dot product.	Memory/work difference.	Treating soft labels as indices.
9. Is the kernel bandwidth-bound?	Shape-dependent: it streams large logits, but exponent/log and reductions can matter; fused design eliminates major intermediate traffic.	Benchmark and roofline.	Universal answer.

Question	Clear answer	Expected key points	Common mistake
10. How do class weights affect backward?	Scale the row's loss/gradient according to the target weighting convention.	Consistency with reduction.	Applying weights only in forward.

7. Deep-Dive Questions

1. **Why is gradient $p-q$?** Differentiating $-\sum q_i(z_i - \log \text{sumexp}(z))$ gives $-q_k + (\sum_i q_i)p_k$; for normalized targets $\sum q=1$, this is $p_k - q_k$.
2. **How do you scale to a huge vocabulary?** Stream/tile each row and use online max/sum state; fuse target gathering and backward where memory constraints justify it.
3. **How can loss reduction become a bottleneck?** Reducing millions of row losses to one scalar needs grid-wide aggregation. Use hierarchical partial sums rather than one highly contended atomic path when scale warrants it.
4. **How does distributed vocabulary parallelism work?** Each device computes local max/sum, all-reduces maxima and appropriately scaled sums, and obtains the target logit from its owner. Communication semantics must preserve stable global log-sum-exp.
5. **Can forward probabilities be discarded?** Yes; backward can recompute softmax from logits/log-sum-exp, trading compute for memory. Training systems choose based on memory pressure and recomputation cost.

8. Comparison Tables

Loss/input path	Stability	Intermediate	Recommended
<code>softmax -> log -> NLL</code>	Can be poor	Full probabilities	No when logits are available
<code>log_softmax -> NLL</code>	Good	Full log probabilities	Good if reused
Fused logits cross entropy	Good	No full probability tensor	Often best
Target type	Storage	Per-row target work	Use case
Sparse class index	One integer	Gather target logit	Standard classification/LM
One-hot dense	C values	Full dot product but redundant	Usually avoid storing
Soft distribution	C values	Full weighted reduction	Distillation/smoothing

9. Common Mistakes

- Taking a logarithm of raw logits.
- Materializing unstable softmax probabilities before the loss.
- Forgetting to add the maximum back in `logsumexp`.
- Averaging over ignored padding tokens.
- Returning p -target without reduction or class-weight scaling.
- Confusing binary cross entropy with multiclass softmax cross entropy.

10. Edge Cases / Special Cases

- Invalid target indices must be rejected or handled by an explicit ignore index.
- All `-inf` logits make normalization undefined without defined behavior.
- Very confident wrong predictions create large finite losses in stable form.
- Empty valid batches make mean reduction undefined; frameworks specify a result/policy.
- Label-smoothing conventions differ on whether mass includes the correct class.
- Distributed vocabulary shards require global, not local, normalization.

11. How to Explain in Interview

"Cross entropy is the negative log probability assigned to the target distribution. With logits, I compute it as stable log-sum-exp minus the target logit, fusing log-softmax and NLL to avoid overflow and a probability intermediate. GPU threads cooperate across classes per row; backward is the normalized probability minus the target, with reduction and weighting applied."

12. Quick Revision Notes

- One-hot loss: `logsumexp(z) - z_target`.
- Stable LSE: `m + log(sum exp(z - m))`.
- Logits gradient: `softmax(z) - q`.
- Ignore index changes both contribution and mean denominator.
- Trap: logits are not probabilities.

13. Practice Tasks

1. Implement naive and stable CPU cross entropy on extreme logits.
2. Verify `p - q` using finite differences.
3. Add sparse targets, label smoothing, weights, and ignore index.
4. Write a block-per-row CUDA fused forward kernel.
5. Compare memory use of separate softmax+NLL and fused cross entropy.

14. Final Cheat Sheet

Item	Must remember
Core definition	Negative expected log predicted probability.
Why it matters	Standard objective for classification and language modeling.
Most asked	Log-sum-exp, <code>p - q</code> , label smoothing, ignore index, fusion.
Main comparison	Fused logits CE is more stable and uses less memory than softmax then log.
One-line answer	"Compute stable log-sum-exp, subtract the target logit, and use <code>softmax-target</code> in backward."

Matrix Multiplication

1. Overview

Definition. Matrix multiplication, or GEMM, computes:

$$C[M,N] = A[M,K] * B[K,N]$$
$$C[i,j] = \text{sum_k } A[i,k] * B[k,j]$$

It is the dominant computation in dense neural networks: linear layers, attention projections, MLPs, convolutions lowered to GEMM, and batched operations. Interviewers ask about it because it exposes the full GPU hierarchy—threads, warps, blocks, memory coalescing, shared-memory tiling, register blocking, Tensor Cores, arithmetic intensity, and boundary handling.

2. Core Idea

Each output cell is a dot product between one row of **A** and one column of **B**. A naive thread can compute one **C[i,j]**, but neighboring threads repeatedly fetch the same input values. Tiling fixes this: a block loads a tile of **A** and a tile of **B** into shared memory, synchronizes, performs many multiply-adds using those reused values, and advances through **K**.

For **A=[[1,2],[3,4]]** and **B=[[5,6],[7,8]]**, **C[0,1]=1*6+2*8=22**. On the GPU:

- 1. Choose an output tile, for example **128x128**.
- 2. Cooperatively load an **A** panel and **B** panel for a **K** slice.
- 3. Multiply them into per-thread register accumulators.
- 4. Repeat for all **K** slices.
- 5. Apply an optional epilogue and store coalesced outputs.

3. Important Subtopics

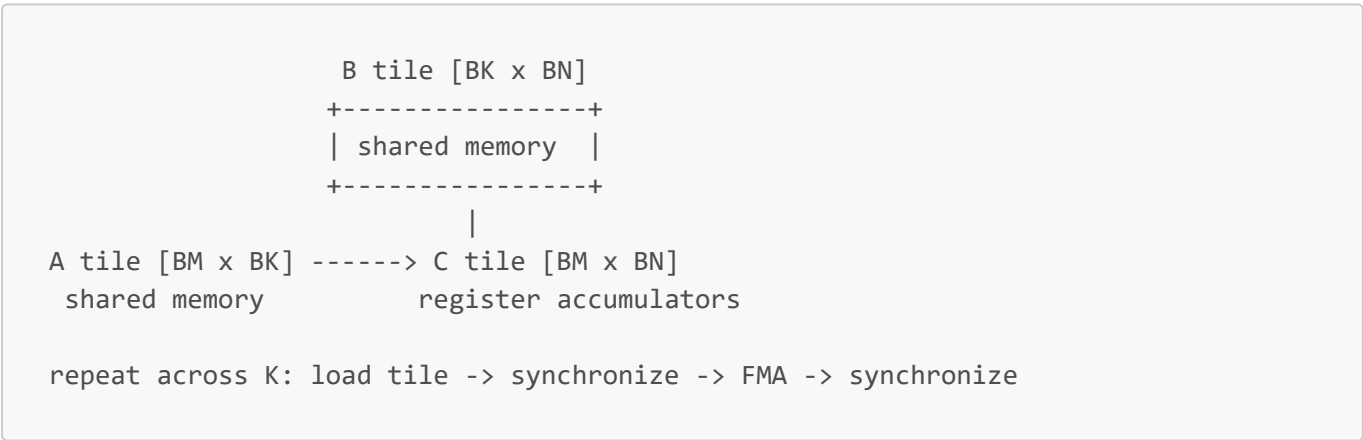
Subtopic	What it means and why it matters	Example	Common interview angle
Thread/output mapping	Decide which output elements each thread owns.	One thread computes a small register tile.	Why not exactly one output per thread?
Global coalescing	Adjacent lanes load/store contiguous addresses.	Warp loads a row segment.	Row-major indexing and transposed operands.
Shared-memory tiling	Reuse loaded A/B values across many FMAs.	BK -wide panels.	Arithmetic intensity improvement.
Bank conflicts	Shared-memory lane addresses may serialize.	Padding/transposed layouts can help.	Diagnose rather than blindly pad.
Register blocking	Each thread accumulates multiple outputs.	TM x TN register tile.	Reuse versus register pressure.

Subtopic	What it means and why it matters	Example	Common interview angle
Double buffering	Load the next tile while computing the current one.	Asynchronous copy pipelines.	Latency hiding and synchronization.
Tensor Cores	Warp-level matrix-multiply-accumulate on tiles.	FP16/BF16 inputs with FP32 accumulation.	Layout/alignment and fragment mapping.
Epilogue fusion	Apply bias, activation, scaling, or quantization before store.	GEMM+bias+GELU.	Avoid extra tensor passes.

4. Real-World Example

A Transformer linear layer multiplies activations `[tokens, K]` by weights `[K, N]`. Production libraries choose kernels based on sizes, layouts, and datatype. Large aligned problems use deeply pipelined Tensor Core tiles; tiny batches may need smaller tiles or persistent kernels because launch latency and insufficient parallelism dominate.

5. Diagrams / Mental Models



Quantity	Approximate value for GEMM
Floating-point work	$2 * M * N * K$ FLOPs
Output elements	$M * N$
Naive input reads	About $2 * M * N * K$ scalar reads without cache reuse
Tiled input per K-slice	About $BM * BK + BK * BN$ values per block

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What does one output contain?	A length- K dot product of an A row and B column.	Reduction dimension.	Multiplying corresponding coordinates only once.
2. Why is naive GEMM slow?	It repeatedly loads inputs with little explicit reuse, lowering arithmetic intensity.	Memory traffic, not lack of threads.	Saying GPUs are bad at loops.
3. How does tiling help?	A block loads input tiles once and reuses them for many output FMAs.	Shared memory and synchronization.	Saying tiling reduces total FLOPs.
4. Why use register tiles?	A thread reuses loaded operands across multiple accumulators.	Higher reuse versus register pressure.	Maximizing tile size without occupancy concern.
5. What are coalesced accesses?	Lanes in a warp access contiguous/aligned memory segments.	Global transaction efficiency.	Confusing coalescing with shared-memory banks.
6. Why synchronize around shared tiles?	Ensure all loads complete before consumption and all consumption completes before overwrite.	Two hazards.	Omitting the second barrier in a single-buffer loop.
7. How do Tensor Cores change the kernel?	They perform warp-cooperative matrix tile operations at high throughput, requiring supported types/layouts and careful data movement.	MMA, mixed precision.	Treating them as one-thread scalar units.
8. What if dimensions are not tile multiples?	Predicate global loads/stores and use zero for invalid reduction elements.	Correct boundary tiles.	Returning entire boundary blocks early.
9. What is arithmetic intensity?	FLOPs per byte transferred from a memory level; tiling raises it through reuse.	Roofline model.	Computing it without stating memory level.
10. Why use cuBLAS instead of a custom kernel?	It contains hardware- and shape-tuned algorithms and is usually faster and safer; custom kernels make sense for fusion or special shapes.	Practical engineering judgment.	Claiming custom code always wins.

7. Deep-Dive Questions

1. **How do tile sizes affect performance?** Larger tiles increase reuse but consume more shared memory/registers, can reduce occupancy, and create more expensive edge waste. Tile shape should match data layout and workload aspect ratio.
2. **What is split-K?** Multiple blocks divide the K reduction for the same output tile, improving parallelism for small M, N and large K ; partial outputs then need atomics or a reduction workspace.
3. **How does double buffering work?** Maintain two shared-memory stages: compute from one while asynchronously filling the other, then synchronize/swap. It overlaps memory latency with arithmetic but uses more shared memory.
4. **Why can more occupancy make GEMM slower?** High-performance GEMM needs registers/shared memory for reuse. Reducing those resources merely to raise resident warps can lower arithmetic intensity and instruction efficiency.
5. **How would you validate mixed-precision GEMM?** Use a high-precision reference, dimensions including edge tiles, structured worst cases, and error bounds that grow with K ; also verify the exact accumulation and conversion semantics.

8. Comparison Tables

Kernel	Data reuse	Complexity	Typical use
Naive one-output/thread	Cache-dependent, low	Low	Teaching/reference
Shared-memory tiled	Block-level	Medium	General CUDA baseline
Register-blocked/pipelined	Block + thread reuse	High	High performance
Tensor Core GEMM	Hardware MMA tiles	High	Supported mixed-precision workloads

Resource	Role	Too little	Too much
Registers	Accumulators/operand fragments	Less reuse	Spills/lower occupancy
Shared memory	Input tile reuse/pipeline	More global traffic	Fewer resident blocks
Threads	Cooperative loads/compute	Poor parallelism	Scheduling/overhead

9. Common Mistakes

- Swapping M, N , and K in indexing or grid dimensions.
- Assuming square, contiguous matrices and ignoring leading dimensions.
- Using shared memory without actual cross-thread reuse.
- Missing a barrier before overwriting a tile.
- Ignoring register spills while increasing per-thread output tiles.
- Comparing a custom kernel to cuBLAS without warm-up and correct synchronization.

10. Edge Cases / Special Cases

- Zero-sized dimensions and $K=0$ require defined output/beta behavior.
- Boundary tiles need masked loads and stores.

- Transposed/non-contiguous operands change coalescing and leading dimensions.
- Aliasing **C** with **A** or **B** is generally unsafe unless explicitly supported.
- Very small matrices are often latency-bound.
- Large **K** increases accumulated floating-point error.

11. How to Explain in Interview

"GEMM computes dot products over **K**. The naive GPU mapping repeats global loads, so an optimized kernel tiles **A** and **B** into shared memory, lets threads accumulate small output tiles in registers, and pipelines later tiles. Coalescing, resource usage, edge predicates, and Tensor Core layouts decide performance; production code normally starts with cuBLAS and customizes for fusion or unusual shapes."

12. Quick Revision Notes

- Work: about **2MNK** FLOPs.
- Core optimization: reuse inputs through shared-memory and register tiling.
- Two tile barriers prevent load-use and overwrite-use hazards.
- Tensor Cores are warp-cooperative and layout-sensitive.
- Trap: maximum occupancy is not the same as maximum performance.

13. Practice Tasks

1. Implement naive one-output-per-thread GEMM and validate rectangular cases.
2. Add shared-memory tiling with non-multiple dimensions.
3. Add a small register output tile and inspect register count/spills.
4. Compare effective TFLOP/s with cuBLAS across square, tall, and skinny shapes.
5. Fuse bias+ReLU/GELU into the output epilogue.
6. Use a profiler to identify global load efficiency, occupancy, and Tensor Core utilization.

14. Final Cheat Sheet

Item	Must remember
Core definition	$C[i,j] = \sum_k A[i,k]B[k,j]$.
Why it matters	Dominant dense-ML compute primitive.
Most asked	Tiling, coalescing, synchronization, Tensor Cores, arithmetic intensity.
Main comparison	Naive streams repeatedly; tiled GEMM reuses data.
One-line answer	"Tile globally loaded operands, reuse them through shared memory/registers, and accumulate each output across K ."

Softmax

1. Overview

Definition. Softmax converts a vector of arbitrary real-valued logits into non-negative probabilities that sum to one:

$$\text{softmax}(x_i) = \exp(x_i) / \sum_j \exp(x_j)$$

It matters because models often need a normalized distribution: class probabilities in a classifier and attention weights in a Transformer. Real systems run softmax over the vocabulary dimension, an attention row, or another chosen axis. Interviewers ask about it because a seemingly simple formula combines parallel reductions, numerical stability, memory bandwidth, warp cooperation, and fusion.

2. Core Idea

Softmax is like turning runners' raw scores into shares of a fixed prize pool. Exponentiation strongly rewards larger scores, then division makes all shares total 1.

For $x = [1, 2, 3]$, direct exponentials are approximately $[2.718, 7.389, 20.086]$; dividing by their sum gives $[0.090, 0.245, 0.665]$. In practice, subtract the maximum first:

```
m = max(x) = 3
e = exp(x - m) = [exp(-2), exp(-1), exp(0)]
s = sum(e)
y = e / s
```

Subtracting the same constant does not change the answer because the factor $\exp(-m)$ cancels. It does prevent $\exp(\text{large_value})$ from overflowing. A GPU row-wise softmax therefore performs: load a row, reduce maximum, compute exponentials, reduce their sum, normalize, and write. Efficient kernels keep reusable values in registers or shared memory and cooperate within a warp or block.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Stable softmax	Subtract the row maximum before exponentiation.	$[1000, 1001]$ becomes $[-1, 0]$.	Prove the result is unchanged and explain overflow prevention.
Row-wise reduction	Maximum and sum require all participating threads to combine values.	One warp handles a short attention row.	Warp shuffle versus shared-memory reduction.
Online softmax	Update max and normalization statistics while streaming tiles.	FlashAttention never materializes a full score row.	Derive the rescaling rule when a new maximum appears.
Masked softmax	Invalid positions act as logits of negative infinity.	Causal attention masks future tokens.	Avoid NaN for fully masked rows.

Subtopic	What it means and why it matters	Example	Common interview angle
Log-softmax	Computes $x_i - \log(\sum(\exp(x)))$ stably.	Used before negative log-likelihood.	Why fused log-softmax is better than $\log(\text{softmax}(x))$.
Precision	FP16/BF16 inputs commonly use FP32 reductions.	Sum thousands of probabilities in FP32.	Accumulator type versus storage type.
Fusion	Combine scale, mask, softmax, dropout, or loss.	Scaled masked softmax in attention.	Reduced kernel launches and memory traffic.
Backward pass	If $g = dL/dy$, then $dL/dx_i = y_i(g_i - \sum_j g_j y_j)$.	One dot-product reduction per row.	Avoid forming the full Jacobian.

4. Real-World Example

In self-attention, each query produces scores against all keys. The scores are scaled by $1/\sqrt{d}$, invalid positions are masked, and softmax converts the row into weights used to blend value vectors. A production kernel often fuses scaling and masking into softmax so the score matrix is read and written fewer times.

5. Diagrams / Mental Models

```

logits row
  |
parallel max reduction -> m
  |
exp(x - m)
  |
parallel sum reduction -> s
  |
divide every element by s -> probabilities

```

Mapping	Best fit	Main mechanism
One warp per row	Short rows	Registers and shuffle reductions
One block per row	Medium rows	Multiple warps plus shared memory
Multiple blocks per row	Very long rows	Staged reductions or persistent/cooperative design

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. Why subtract the maximum?	It prevents positive exponent overflow while preserving the distribution.	Algebraic invariance; largest	Saying it prevents every possible underflow; tiny terms

Question	Clear answer	Expected key points	Common mistake
		exponent becomes 1.	may safely underflow to zero.
2. Why are two reductions needed?	One finds the maximum; the other sums stabilized exponentials.	Max, then sum; both are row-wide dependencies.	Computing the sum before stabilization.
3. Is softmax compute- or bandwidth-bound?	Usually bandwidth/latency-bound for ordinary row sizes because each element has little reusable work, though exponent throughput and shape can matter.	Roofline reasoning; measure on target hardware.	Giving one universal classification.
4. How should threads map to data?	Adjacent lanes should load adjacent elements; one warp/block commonly owns a row.	Coalescing and row ownership.	One thread computes an entire long row.
5. Why accumulate in FP32?	Reductions amplify rounding error and low-precision exponent/sum may lose small contributions.	Mixed precision.	Assuming FP16 Tensor Cores accelerate exponentials.
6. How is masking implemented?	Replace masked logits logically with <code>-inf</code> before the max and sum.	Mask precedes normalization.	Setting masked probabilities to zero only after softmax, which changes the denominator.
7. What is the backward formula?	$dx = y * (dy - \text{sum}(dy*y))$, elementwise except for one row reduction.	Jacobian-vector product, not explicit Jacobian.	Treating each output as independent.
8. Softmax versus sigmoid?	Softmax couples a vector into a distribution summing to one; sigmoid independently maps each scalar to $(0,1)$.	Multiclass versus multilabel.	Claiming sigmoid outputs always sum to one.
9. What is online softmax?	A streaming algorithm that maintains a running max and rescaled exponential sum.	Rescale old sum when max changes.	Adding tile sums without reconciling different maxima.
10. What can be fused?	Scale, bias/mask, dropout, and sometimes cross-entropy or attention value accumulation.	Avoid intermediates and launches.	Fusing so much that register pressure reduces occupancy without measuring.

7. Deep-Dive Questions

1. **Derive the online update.** For old state (m, l) and a new tile with maximum m_t and sum $l_t = \text{sum}(\exp(x-m_t))$, set $m_{\text{new}} = \max(m, m_t)$ and $l_{\text{new}} = l * \exp(m - m_{\text{new}}) + l_t * \exp(m_t - m_{\text{new}})$. Both sums

are moved to the new reference maximum.

2. **How do you handle rows larger than one block?** Use a multi-stage reduction, cooperative launch, or a streaming block that processes several chunks. The choice trades launch overhead, synchronization scope, and parallelism.
3. **Why can a fully masked row produce NaN?** Its maximum is `-inf`; evaluating `-inf - (-inf)` is undefined. Define semantics explicitly—often output zeros—and branch or sanitize that row.
4. **Could atomics implement the reductions?** Yes, but usually with contention and awkward ordering between max and sum. Hierarchical warp/block reductions are normally faster and deterministic enough for the operation.
5. **How would you benchmark a softmax kernel?** Sweep row count, row width, datatype, masks, and alignment; compare correctness to a high-precision reference; report effective bytes/s, latency, and end-to-end fused performance.

8. Comparison Tables

Variant	Output	Stability	Typical use
Naive softmax	Probabilities	Poor for large logits	Teaching only
Stable softmax	Probabilities	Good	Standard inference/training
Log-softmax	Log probabilities	Good	Cross-entropy/NLL
Online softmax	Probabilities or running weighted result	Good	Tiled/streaming attention

Reduction method	Scope	Synchronization	Good for
Warp shuffle	32 lanes	Warp-synchronous	Short rows
Shared-memory tree	Block	<code>__syncthreads()</code>	Medium/long rows
Multi-kernel reduction	Grid	Kernel boundary	Extremely long rows

9. Common Mistakes

- Computing `exp(x)` before finding and subtracting the maximum.
- Applying a mask after normalization.
- Assuming one warp is always sufficient regardless of row length.
- Reading the row from global memory separately for max, sum, and output when values could be retained.
- Using exact equality to validate floating-point output.
- Constructing the softmax Jacobian in backward instead of using a vector-Jacobian product.

10. Edge Cases / Special Cases

- A one-element row returns `1` unless it is masked under special semantics.
- Very negative differences may underflow to zero; this is normally harmless.
- `+inf`, `-inf`, or `NaN` inputs require a documented policy.

- Non-contiguous axes need stride-aware addressing or a prior transpose.
- Rows not divisible by warp/tile size need predicated loads with neutral elements (`-inf` for max, `0` for sum).
- Fully masked rows need explicit handling.

11. How to Explain in Interview

"Softmax turns a row of logits into a probability distribution. A stable GPU kernel subtracts the row maximum, performs a parallel sum of exponentials, and normalizes. I map a warp or block to each row, use coalesced loads and shuffle/shared-memory reductions, accumulate in FP32 for low-precision inputs, and fuse masks or adjacent operations when that saves memory traffic."

12. Quick Revision Notes

- Formula: $y_i = \exp(x_i - m) / \sum_j \exp(x_j - m)$, $m = \max(x)$.
- Dependencies: one max reduction and one sum reduction.
- Backward: $dx_i = y_i(dy_i - \sum_j dy_j * y_j)$.
- Main optimization: cooperative reduction, value reuse, and fusion.
- Trap: masked values must be excluded before the denominator is formed.

13. Practice Tasks

1. Implement CPU naive and stable softmax; test logits near ± 1000 .
2. Write a warp-per-row CUDA kernel using shuffle reductions.
3. Extend it to arbitrary row widths and masked tails.
4. Implement and verify the online (`max`, `sum`) update over random tiles.
5. Fuse causal masking and compare global-memory traffic with separate kernels.
6. Derive and implement the backward vector-Jacobian product.

14. Final Cheat Sheet

Item	Must remember
Core definition	Exponentiate and normalize a vector into probabilities.
Why it matters	Classification and attention normalization.
Most asked	Stability, reductions, masking, backward, online softmax.
Main comparison	Stable versus online: same math, full-row versus streaming state.
One-line answer	"Subtract max, reduce exponentials, normalize, and keep the row cooperative and numerically stable."

LayerNorm

1. Overview

Definition. Layer normalization standardizes each sample across its feature dimension, then applies learned per-feature scale and shift:

```
mu    = mean(x)
var   = mean((x - mu)^2)
y_i   = (x_i - mu) / sqrt(var + epsilon) * gamma_i + beta_i
```

Unlike BatchNorm, it does not depend on other batch elements. It is central to Transformers, language models, vision transformers, and many sequence models. Interviewers ask about LayerNorm because it combines reductions, mixed-precision accuracy, fusion, backward derivation, and bandwidth-sensitive GPU implementation.

2. Core Idea

Imagine comparing exam scores from subjects with different scales. First center a student's feature vector around its own average, then scale it by its spread. **gamma** and **beta** let the model restore or reshape useful scales afterward.

For $x=[1,2,3]$, $\mu=2$, $\text{var}=(1+0+1)/3=2/3$. Ignoring affine parameters, the normalized row is approximately $[-1.225, 0, 1.225]$. A GPU kernel commonly assigns one block to a row: threads load features, reduce sum and squared sum (or Welford state), compute inverse standard deviation, then normalize and write. Because each element has little arithmetic relative to bytes moved, fused residual-add + LayerNorm is common.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Normalization axis	Statistics are computed per sample across hidden features.	Transformer tensor $[B,S,H]$ normalizes each (b,s) row over H .	Contrast with BatchNorm.
Variance algorithm	$E[x^2] - E[x]^2$ is fast but can suffer cancellation; Welford is more stable.	Values clustered around a large mean.	Accuracy versus instruction cost.
Epsilon	Added inside the square root to avoid division by zero.	Constant row has variance zero.	Placement and datatype of epsilon.
Affine transform	Learned gamma and beta preserve representational flexibility.	$y = \text{norm} * \text{gamma} + \text{beta}$.	Their shapes and gradients.
Pre-norm/post-norm	LayerNorm occurs before or after a Transformer sublayer.	Modern LLMs often use pre-norm variants.	Training stability, not kernel formula.
Fusion	Residual addition, bias, dropout, and normalization	$\text{LayerNorm}(x + \text{residual})$.	Memory traffic versus register

Subtopic	Meaning and why it matters	Example	Common interview angle
	can share data.		pressure.
Backward	Needs row reductions plus dgamma/dbeta reductions across rows.	Gradients for input and affine parameters.	Avoid per-element atomics when possible.
Mixed precision	Inputs/outputs may be FP16/BF16 while statistics use FP32.	Long hidden dimension.	Numerical stability.

4. Real-World Example

In a Transformer block, the residual stream has shape **[batch, sequence, hidden]**. Each token vector is normalized independently. Fusing residual addition with LayerNorm avoids writing the summed vector to global memory and immediately reading it again, often a larger win than reducing a few arithmetic instructions.

5. Diagrams / Mental Models



State in Welford combine	Meaning
count	Number of values represented
mean	Running mean
M2	Sum of squared deviations from the mean
Final variance	M2 / count for population variance

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What axis does LayerNorm use?	The feature/hidden dimension of each sample, independent of batch peers.	Per-row statistics.	Saying it normalizes each feature across the batch.
2. Why add epsilon?	It prevents division by zero and limits amplification when variance is tiny.	Inside <code>sqrt(var+eps)</code> .	Treating epsilon as a learned value by default.
3. LayerNorm versus BatchNorm?	LayerNorm uses per-sample feature statistics and behaves the same in training/inference; BatchNorm uses batch statistics and running estimates.	Batch dependence and axes.	Only comparing formulas.
4. Why use FP32 statistics?	Sums and variance are sensitive to rounding/cancellation over many features.	Mixed-precision accumulation.	Assuming output datatype must also be FP32.
5. One-pass or two-pass variance?	Two-pass is straightforward and stable; Welford supports stable streaming/parallel combine; sum/sumsq can be faster but less robust.	Shape/hardware tradeoff.	Claiming a single method is universally best.
6. Is LayerNorm bandwidth-bound?	Usually yes: it reads input and affine parameters and writes output with modest arithmetic.	Fusion and bytes moved.	Ignoring expensive reductions or small-shape latency.
7. What does one block usually process?	One or sometimes several rows, with lanes striding through features.	Coalescing and block reductions.	One thread per row for large <code>H</code> .
8. What are <code>gamma</code> and <code>beta</code> ?	Learned vectors of length <code>H</code> that scale and shift normalized features.	Broadcast across rows.	Calling them row-wise scalars.
9. Why fuse residual addition?	The sum can feed statistics directly without an intermediate global-memory round trip.	Bandwidth saving.	Assuming fusion is always faster despite occupancy/register costs.
10. How does backward differ from forward?	Input gradient uses row statistics/reductions; <code>dgamma</code> and <code>dbeta</code> reduce contributions across all rows.	Two reduction directions.	Forgetting parameter gradients.

7. Deep-Dive Questions

1. **How are Welford states combined in parallel?** For states $(n_a, mean_a, M2_a)$ and $(n_b, mean_b, M2_b)$, use $\delta = mean_b - mean_a$, $n = n_a + n_b$, $mean = mean_a + \delta * n_b / n$, and $M2 = M2_a + M2_b + \delta^2 * n_a * n_b / n$.
2. **Why can $E[x^2] - E[x]^2$ fail?** It subtracts two large, nearly equal rounded values when variance is small relative to the mean, causing catastrophic cancellation or even a small negative result.
3. **How would you handle hidden sizes larger than one block can efficiently cover?** Let each thread process multiple vectorized elements and reduce partial states; for extremely large rows, use staged reductions or persistent strategies.
4. **What makes backward parameter gradients difficult?** $d\gamma$ and $d\beta$ reduce across potentially millions of rows. Global atomics are simple but may contend; staged block reductions improve throughput at the cost of workspace/another launch.
5. **What should a fused residual-LayerNorm API return?** Training may need the residual sum, mean, and inverse standard deviation for backward. Saving them increases memory traffic but avoids recomputation; decide from the actual training graph.

8. Comparison Tables

Property	LayerNorm	RMSNorm	BatchNorm
Centers by mean	Yes	No	Yes
Scaling statistic	Variance	Mean square	Batch variance
Depends on batch peers	No	No	Yes
Learned parameters	Usually scale + bias	Usually scale	Scale + bias
Train/inference behavior	Same	Same	Usually different

Variance strategy	Accuracy	Pass/State	Typical tradeoff
Two-pass mean then variance	High	Two scans	More reads unless values retained
Welford	High	Mergeable state	More arithmetic
Sum and sum-of-squares	Lower for difficult inputs	Simple reductions	Fast and compact

9. Common Mistakes

- Normalizing over batch or sequence instead of hidden features.
- Omitting γ and β when describing the full layer.
- Using low-precision accumulation for mean and variance.
- Confusing population variance $(/H)$ with sample variance $(/(H-1))$.
- Assuming epsilon fixes all cancellation in a poor variance algorithm.
- Recomputing or rereading values unnecessarily in a bandwidth-bound kernel.

10. Edge Cases / Special Cases

- Constant rows produce zero normalized values before affine transform.
- $H=1$ has zero variance and output normally becomes β .

- Tail features need masks for vectorized loads and reductions.
- Tiny variance makes the exact epsilon convention observable.
- Non-contiguous tensors require stride-aware kernels.
- NaN in a row normally propagates through its statistics.

11. How to Explain in Interview

"LayerNorm normalizes each sample across its hidden features, then applies learned scale and bias. On a GPU I map a block to a row, reduce stable FP32 statistics, broadcast the inverse standard deviation, and normalize with coalesced/vectorized accesses. Since it is usually bandwidth-bound, fusing residual or bias operations is often the main optimization."

12. Quick Revision Notes

- Formula: $(x - \text{mean}) / \sqrt{\text{var} + \text{eps}} * \text{gamma} + \text{beta}$.
- Axis: hidden dimension per sample; no batch dependency.
- Stable statistic: Welford or a careful two-pass calculation.
- Common mapping: one cooperative block per row.
- Trap: $E[x^2] - E[x]^2$ can lose precision.

13. Practice Tasks

1. Implement CPU LayerNorm and compare sum/sumsq with Welford on large-offset data.
2. Write a block-per-row CUDA forward kernel with FP32 reductions.
3. Add vectorized loads while safely handling a non-multiple tail.
4. Fuse residual addition and measure bytes transferred.
5. Derive input, gamma, and beta gradients and validate numerically.

14. Final Cheat Sheet

Item	Must remember
Core definition	Per-row centering and variance scaling plus learned affine transform.
Why it matters	Stabilizes Transformer activations independently of batch size.
Most asked	Axis, Welford, epsilon, BatchNorm comparison, fusion, backward.
Main comparison	LayerNorm centers and scales; RMSNorm only scales by RMS.
One-line answer	"Reduce per-token FP32 statistics, normalize the hidden row, then apply gamma and beta."

RMSNorm

1. Overview

Definition. RMSNorm scales a feature vector by its root mean square, usually followed by a learned per-feature weight:

```
rms(x) = sqrt(mean(x_i^2) + epsilon)
y_i    = x_i / rms(x) * gamma_i
```

It does not subtract the mean and usually has no bias. Many modern language models use it because it is simpler than LayerNorm while still controlling activation scale. Interviewers ask about it to test whether candidates understand normalization semantics, reductions, numerical precision, and why deleting mean-centering changes both math and kernel cost.

2. Core Idea

RMSNorm is like adjusting an audio signal to a consistent power level without removing its DC offset. It measures magnitude, not how far values lie from their average.

For $x=[3,4]$, mean square is $(9+16)/2=12.5$, RMS is about 3.536 , and the normalized vector is $[0.849, 1.131]$ before learned scaling. A GPU block reduces the squared values, computes one reciprocal RMS, and lets every thread scale its elements. Compared with LayerNorm, there is no sum reduction for the mean and no subtraction during normalization.

3. Important Subtopics

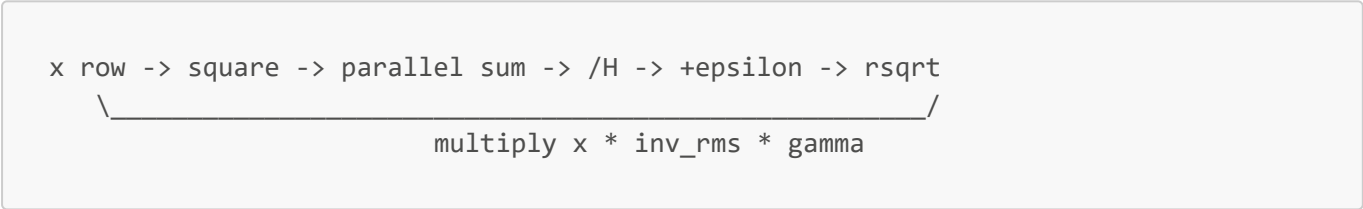
Subtopic	What it means and why it matters	Example	Common interview angle
Mean square	Average of squared features, generally accumulated in FP32.	$\text{sum}(x*x)/H$.	Distinguish from variance.
No centering	The vector's mean is not removed.	Adding a constant changes RMSNorm differently from LayerNorm.	Is RMSNorm shift-invariant? No.
Learned scale	$\text{gamma}[H]$ rescales each feature.	Broadcast across tokens.	Typical absence of beta .
Reciprocal square root	Compute $\text{inv_rms} = \text{rsqrt}(\text{mean_square}+\text{eps})$ once per row.	All lanes reuse it.	Accuracy/performance of rsqrt .
Partial RMSNorm	Estimate RMS from a subset of features.	Research variant, uncommon in basic production paths.	Statistical/performance tradeoff.
Fusion	Residual addition and quantization can be fused with normalization.	LLM inference pipelines.	Bandwidth savings.

Subtopic	What it means and why it matters	Example	Common interview angle
Backward	Couples features through the squared-norm reduction.	Requires a dot-product-like row reduction.	Not purely elementwise.
Overflow control	Squaring large low-precision values can overflow before accumulation.	Convert to FP32 before $x*x$.	Cast timing matters.

4. Real-World Example

An LLM decoder applies RMSNorm to each token's residual vector before attention and MLP projections. During inference, a fused kernel can read the residual once, compute FP32 mean square, scale by γ , and optionally convert the output to the lower-precision format consumed by the next matrix multiplication.

5. Diagrams / Mental Models



Property	Effect
Scale x by positive a	Normalized direction is approximately unchanged
Add constant c	Output changes; there is no mean subtraction
All zeros	$inv_rms=1/\sqrt{eps}$, but output remains zero before any bias

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What does RMSNorm compute?	It divides each feature by the row's root mean square and applies learned scale.	Mean square, epsilon, gamma.	Calling mean square "variance."
2. How does it differ from LayerNorm?	It omits mean subtraction and usually bias, requiring fewer operations/reductions.	No centering; semantic difference.	Saying they are mathematically identical.
3. Why can it be faster?	It removes the mean statistic and subtraction, and has a simpler backward path, though memory traffic may still dominate.	Kernel is often bandwidth-bound.	Promising a fixed speedup.

Question	Clear answer	Expected key points	Common mistake
4. What axis is normalized?	Usually the hidden/features dimension for each token or sample.	Same common axis as LayerNorm.	Reducing over batch.
5. Why FP32 accumulation?	Squaring and summing many low-precision values can overflow or accumulate large error.	Cast before multiply.	Multiplying in FP16 then casting the product.
6. What happens for a zero row?	The output is zero because the numerator is zero, despite finite <code>inv_rms</code> .	Epsilon prevents division by zero.	Claiming output becomes <code>1/sqrt(eps)</code> .
7. Is it elementwise?	The final scale is elementwise, but computing RMS is a row-wide reduction.	Coupling through statistic.	Describing it as only a multiply.
8. What is the kernel mapping?	Usually one block per row, threads striding across features and reducing squared values.	Coalescing and reduction.	One block per element.
9. Can it be fused?	Yes, especially with residual add, bias, type conversion, or the next operation's input preparation.	Avoid intermediate memory traffic.	Ignoring saved values needed for backward.
10. Is RMSNorm shift-invariant?	No; adding a constant changes its denominator and numerator because it does not center.	Contrast with LayerNorm's centering behavior.	Confusing scale invariance with shift invariance.

7. Deep-Dive Questions

- Derive the input gradient.** Let $r = \text{rsqrt}(\text{mean}(x^2) + \text{eps})$ and $u = dy * \text{gamma}$. Then $dx_i = r * u_i - x_i * r^3 * (\text{sum}_j u_j * x_j) / H$; the second term explains the row reduction.
- How could you avoid overflow while computing RMS?** Convert inputs to FP32 before squaring; for extreme ranges, use a scaled sum-of-squares algorithm that tracks the largest magnitude, similar to stable vector norm computation.
- When might RMSNorm not replace LayerNorm?** When mean-centering is important to the learned architecture or compatibility with pretrained weights. Kernel simplicity cannot justify changing model semantics.
- Would Tensor Cores help?** Not directly for the row reduction and elementwise scaling; ordinary CUDA cores, vector loads, and bandwidth optimization dominate.
- How do you validate a fused low-precision implementation?** Compare to an FP64/FP32 reference across zero, constant, high-dynamic-range, odd-sized, and random rows using error tolerances and end-to-end model checks.

8. Comparison Tables

Feature	RMSNorm	LayerNorm
Statistic	$\text{mean}(x^2)$	$\text{mean}(x)$ and $\text{mean}((x - \text{mean})^2)$
Mean subtraction	No	Yes
Typical learned params	γ	γ, β
Shift-invariant	No	Approximately yes before affine transform
Reduction work	One sum of squares	Mean plus variance statistics
Implementation	Benefit	Cost/risk
Scalar loads	Simple	More instructions
Vectorized loads	Higher memory efficiency	Alignment and tail handling
Fused residual RMSNorm	Avoids intermediate	More registers and graph coupling

9. Common Mistakes

- Calling RMS a standard deviation.
- Subtracting the mean, accidentally implementing LayerNorm.
- Squaring in FP16 before converting to FP32.
- Forgetting division by hidden size.
- Assuming omission of one statistic makes the kernel compute-bound.
- Adding a bias merely because LayerNorm has one.

10. Edge Cases / Special Cases

- Zero and near-zero rows are dominated by epsilon.
- Very large magnitudes can overflow during squaring if conversion happens too late.
- Odd hidden sizes need predication for vector loads.
- The exact epsilon value is part of model compatibility.
- Some implementations place epsilon differently; weights/checkpoints expect the original convention.
- Non-finite inputs normally propagate.

11. How to Explain in Interview

"RMSNorm scales each token vector by the reciprocal square root of its mean squared magnitude and applies a learned feature scale. Unlike LayerNorm it does not subtract the mean. A GPU block reduces squares in FP32, broadcasts one inverse RMS, and performs coalesced elementwise scaling, often fused with a residual add."

12. Quick Revision Notes

- Formula: $x * \text{rsqrt}(\text{mean}(x^2) + \text{eps}) * \gamma$.
- No centering; mean square is not variance.
- One row-wide sum reduction.
- Convert before squaring for mixed precision.
- Trap: simpler/faster does not mean interchangeable with LayerNorm.

13. Practice Tasks

1. Compare RMSNorm and LayerNorm on vectors with the same variance but different means.
2. Implement a block-per-row CUDA forward kernel.
3. Add FP16 input/FP32 accumulation and test high-magnitude values.
4. Derive backward and verify with finite differences.
5. Fuse residual addition and profile memory throughput.

14. Final Cheat Sheet

Item	Must remember
Core definition	Scale a row by its root mean square, then by learned gamma.
Why it matters	Cheap, batch-independent normalization in many LLMs.
Most asked	LayerNorm difference, precision, reduction, fusion, backward.
Main comparison	RMSNorm measures magnitude; LayerNorm measures centered spread.
One-line answer	"Reduce FP32 squares per row, compute one inverse RMS, and scale all features without mean subtraction."

Attention

1. Overview

Definition. Scaled dot-product attention lets every query select and combine information from keys and values:

```
S = QK^T / sqrt(d_k)
P = softmax(S + mask)
O = PV
```

For $Q: [B, H, N_q, d]$, $K: [B, H, N_k, d]$, and $V: [B, H, N_k, d_v]$, output is $[B, H, N_q, d_v]$. Attention drives Transformers in language, vision, audio, and multimodal systems. Interviewers ask about it because it combines GEMM, masking, softmax, tensor layout, quadratic complexity, batching, cache behavior, and opportunities for fusion.

2. Core Idea

Think of a query as a search request, keys as item descriptions, and values as the information returned. A dot product scores how well a query matches each key. Softmax turns scores into weights, and the weighted sum of values returns the answer.

For one query with scores $[2, 1, 0]$, scaled/stable softmax might produce roughly $[0.665, 0.245, 0.090]$. If values are vectors v_0, v_1, v_2 , the output is $0.665v_0 + 0.245v_1 + 0.090v_2$.

Step by step:

1. Project input states into Q , K , and V with GEMMs.
2. Compute all query-key dot products.
3. Scale by $1/\sqrt{d}$ to control score variance.
4. Add causal/padding bias so invalid positions receive no probability.
5. Normalize each query row with softmax.
6. Multiply the probabilities by V .

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Scaling	Divide scores by $\sqrt{d_k}$.	Prevents dot products from growing with head dimension.	Derive variance intuition.
Multi-head attention	Split representation into independent attention heads.	$d_{model}=4096$, 32 heads of 128.	Shapes and parallel axes.
Causal mask	Query i cannot attend to keys $j > i$.	Autoregressive decoding.	Mask before softmax.
Padding/variable length	Ignore padded tokens or use sequence metadata.	Batched sentences.	Avoid wasted computation with packed layouts.
MHA/MQA/GQA	Number of key/value heads may equal or be fewer than query heads.	GQA shares K/V among groups.	KV-cache memory reduction.
KV cache	Store prior keys/values during autoregressive inference.	Decode one token at a time.	Bandwidth and layout dominate decode.
Dropout	Randomly masks/scales attention probabilities in training.	Needs reproducible RNG mapping.	Fusion and backward state.
Backward	Gradients pass through PV , softmax, and QK^T .	Several GEMMs plus reductions.	Softmax Jacobian-vector product.

4. Real-World Example

During LLM decoding, a new token creates one query per head and attends to thousands of cached keys/values. Unlike training prefill, the query dimension is tiny, so there is limited GEMM parallelism and each cached element may be read only once. The operation becomes strongly memory-bandwidth and latency sensitive; MQA/GQA reduces KV-cache traffic by sharing key/value heads.

5. Diagrams / Mental Models

```

Input X
|--- projection ---> Q ----\
|--- projection ---> K ----- dot/scale/mask -> softmax --\
|--- projection ---> V ----- weighted sum -> O

```

Stage	Shape (single batch/head)	Main operation
Scores	$[N_q, d] \times [d, N_k] \rightarrow [N_q, N_k]$	GEMM
Probabilities	Each row of $[N_q, N_k]$	Masked softmax
Output	$[N_q, N_k] \times [N_k, d_v]$	GEMM

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What does attention compute?	Query-key similarities become normalized weights used to combine values.	$\text{softmax}(QK^T/\sqrt{d})V$.	Saying values are part of the score dot product.
2. Why divide by $\sqrt{d_k}$?	If components have unit variance, dot-product variance grows with d ; scaling keeps logits in a useful range.	Softmax saturation/gradients.	Saying it only prevents numerical overflow.
3. What is causal attention?	Position i can attend only to positions at or before i .	Upper-triangular mask before softmax.	Zeroing outputs after softmax.
4. What is the complexity?	Standard dense self-attention has $O(N^2d)$ arithmetic and an $O(N^2)$ score/probability intermediate if materialized.	Separate compute and memory complexity.	Saying FlashAttention changes exact attention to $O(N)$.
5. Why multiple heads?	Heads learn different subspaces/relationships and operate independently before concatenation/projection.	Shape reasoning.	Claiming heads attend to fixed linguistic roles by design.
6. MHA versus MQA/GQA?	MHA has K/V per query head; MQA has one shared K/V head; GQA shares within groups.	KV-cache bandwidth/capacity.	Saying query heads are also collapsed.
7. What is the KV cache?	Stored keys and values from previous tokens so decoding does not recompute them.	Append/read, not cached queries.	Saying it removes attention over previous tokens.

Question	Clear answer	Expected key points	Common mistake
8. Where is softmax applied?	Across the key dimension independently for each query, batch, and head.	Row-wise axis.	Across heads or feature dimension.
9. Why can attention be memory-heavy?	Materialized scores/probabilities scale as $N_q * N_k$ and are read/written between stages.	Intermediate traffic.	Counting only Q/K/V tensors.
10. How do padding masks differ from causal masks?	Padding masks reflect valid sequence lengths; causal masks enforce temporal ordering. They may be combined.	Different semantics.	Treating padding as always triangular.

7. Deep-Dive Questions

- Why is decode attention different from prefill?** Prefill has many queries and large matrix multiplications; decode has one/few queries, low reuse of the large KV cache, and is commonly bandwidth/latency-bound.
- How are attention gradients structured?** From $O=PV$, compute $dV=P^T dO$ and $dP=dO V^T$; softmax backward gives $dS=P*(dP-\text{row_sum}(dP*P))$; then $dQ=dS K*scale$ and $dK=dS^T Q*scale$.
- What layout is best for KV cache?** It depends on decode kernel access, head grouping, vector width, paging, and cache growth. The key requirement is contiguous/coalesced access along the dimension lanes consume, without expensive per-step transposes.
- How do relative/rotary positions interact with attention?** They modify queries/keys or scores before softmax but do not change the weighted-value definition. Fusion can avoid extra Q/K tensor passes.
- When is sparse/local attention useful?** When the application tolerates a restricted connectivity pattern, reducing actual work and memory. It changes model semantics, unlike exact tiled attention.

8. Comparison Tables

Variant	Q heads	KV heads	KV-cache size	Typical benefit
MHA	H	H	Highest	Maximum per-head flexibility
GQA	H	Between 1 and H	Lower	Quality/performance compromise
MQA	H	1	Lowest	Decode bandwidth reduction

Phase	Query length	Dominant concern	Common optimization
Training/prefill	Many	Compute plus quadratic intermediates	Fused tiled attention
Autoregressive decode	One/few	KV-cache bandwidth and launch latency	GQA/MQA, paging, decode-specialized kernel

9. Common Mistakes

- Applying softmax along the head or feature dimension.
- Masking after softmax.
- Omitting $1/\sqrt{d_k}$ or applying it to the output.
- Claiming all attention optimization changes quadratic arithmetic complexity.
- Confusing KV cache with caching attention probabilities.
- Ignoring batch, head, and stride dimensions when deriving layouts.

10. Edge Cases / Special Cases

- Fully masked query rows need explicit semantics to avoid NaN.
- N_q and N_k can differ in cross-attention.
- Head dimension may not match vector/Tensor Core tile sizes.
- Variable-length batches need masks, packed sequences, or per-sequence metadata.
- Dropout appears in training, not ordinary inference.
- Very long decode contexts may use paged/non-contiguous KV storage.

11. How to Explain in Interview

"Attention scores each query against keys, scales and masks those scores, normalizes across keys, then uses the probabilities to blend values. The basic implementation is two GEMMs around a row-wise softmax, but materializing the quadratic score matrix creates heavy memory traffic. Training/prefill favors fused tiled kernels; token-by-token decode is often dominated by reading the KV cache."

12. Quick Revision Notes

- Formula: $\text{softmax}(QK^T/\sqrt{d} + \text{mask})V$.
- Softmax axis: keys for each query/head.
- Dense work: $O(N_q * N_k * d)$; materialized scores: $O(N_q * N_k)$.
- MQA/GQA reduce KV-cache size and bandwidth.
- Trap: causal mask must enter before softmax.

13. Practice Tasks

1. Trace all tensor shapes for multi-head self- and cross-attention.
2. Implement a CPU attention reference with causal and padding masks.
3. Build attention from GEMM + softmax + GEMM library calls.
4. Estimate score-matrix bytes for several sequence lengths and datatypes.
5. Compare prefill and decode arithmetic intensity.
6. Derive and numerically check the backward formulas.

14. Final Cheat Sheet

Item	Must remember
Core definition	Normalize query-key scores and use them to weight values.
Why it matters	Context mixing mechanism in Transformers.

Item	Must remember
Most asked	Scaling, mask, shapes, complexity, KV cache, MHA/GQA/MQA.
Main comparison	Prefill is GEMM-rich; decode streams a large KV cache for few queries.
One-line answer	"Score Q against K, stable-softmax over keys, and multiply the weights by V."

FlashAttention-Style Tiled Attention

1. Overview

Definition. FlashAttention-style attention computes exact scaled dot-product attention in tiles without writing the full score or probability matrix to high-bandwidth memory (HBM). It uses online softmax statistics so each query tile can consume key/value tiles incrementally.

It matters because ordinary attention's large intermediate tensors cause enormous memory traffic. FlashAttention-style kernels are used for Transformer training and prefill, especially at long sequence lengths. Interviewers ask about them to test GPU memory hierarchy, loop tiling, online numerical stability, fusion, recomputation, and the distinction between arithmetic complexity and I/O complexity.

2. Core Idea

Standard attention often performs:

```
HBM: Q,K -> scores -> HBM -> softmax -> probabilities -> HBM -> multiply V -> O
```

Tiled attention instead keeps a query tile and its output accumulator on chip. It streams key/value tiles, computes one score tile, updates row-wise online softmax state, immediately multiplies the current probabilities into the value tile, and discards the score tile.

For each query row maintain:

```
m = running maximum score
l = running sum of exp(score - m)
o = running weighted value numerator under the same scale
```

When a new tile has maximum m_t , choose $m_{\text{new}} = \max(m, m_t)$. Rescale the old state by $\alpha = \exp(m - m_{\text{new}})$ and the new tile by its reference to m_{new} ; then update l and o . At the end output o/l . This is exact apart from floating-point order, not an approximation or sparse-attention method.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
IO awareness	Optimize transfers between HBM and on-chip memory.	Never store $N \times N$ probabilities.	Complexity is about memory traffic, not fewer attention pairs.
Q/K/V tiling	Load small blocks that fit shared memory/registers.	Keep a Q block while streaming K/V blocks.	Choose loop order and reuse.
Online softmax	Merge tile statistics under a common running max.	Rescale old numerator when max grows.	Derive (m, l, o) update.
On-chip residency	Scores/probabilities live briefly in registers/shared memory.	Consume P_tile @ V_tile immediately.	Resource limits determine tile sizes.
Causal skipping	Tiles wholly above the causal diagonal can be skipped; diagonal tiles use element masks.	Less work in causal attention.	Block-level versus element-level mask.
Backward recomputation	Recompute score/probability tiles rather than store the full matrix.	Save output and log-sum-exp statistics.	More FLOPs, much less memory.
Pipelining	Overlap asynchronous K/V tile loads with MMA computation.	Multi-stage shared-memory buffers.	Barriers and resource pressure.
Numerical precision	Scores/output accumulators and softmax stats commonly use FP32.	FP16/BF16 operands, FP32 accumulation.	Stable merges across tiles.

4. Real-World Example

For sequence length 8192, one attention matrix per head contains about 67 million elements. In FP16 that is about 128 MiB for scores alone, before probabilities, gradients, heads, or batch. A fused tiled kernel avoids these quadratic HBM intermediates, so longer contexts become practical even though it still evaluates the required dense query-key interactions.

5. Diagrams / Mental Models

Keep Q tile + (m, l, o) accumulator) on chip

```

K tile 0 + V tile 0 -> scores -> online softmax -> update 0
K tile 1 + V tile 1 -> scores -> online softmax -> rescale/update 0
K tile 2 + V tile 2 -> scores -> online softmax -> rescale/update 0
...
final 0 = accumulated numerator / 1 -> one HBM write

```

Data	Standard materialized path	FlashAttention-style path
Q, K, V	Read	Read in tiles
Full scores S	Write then read	Never written to HBM
Full probabilities P	Write then read	Never written to HBM
Output O	Write	Write

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What problem does FlashAttention solve?	Excessive HBM traffic and storage from materialized attention matrices.	IO-aware exact attention.	Saying it primarily reduces asymptotic FLOPs.
2. Is it approximate?	No, standard FlashAttention computes exact dense attention up to floating-point ordering.	Same mathematical result.	Confusing it with sparse/linear attention.
3. Why is online softmax necessary?	A score row is seen tile by tile, so normalization must be updated without seeing/storing the full row.	Running max, sum, numerator.	Normalizing each tile independently and concatenating.
4. Why rescale old accumulators?	A later tile may introduce a larger maximum, changing the exponential reference for all earlier contributions.	$\exp(m_{old} - m_{new})$.	Updating only the denominator.
5. What stays on chip?	Query tile, softmax statistics, output accumulators, and current K/V or score fragments.	Registers/shared memory.	Claiming the entire sequence fits in shared memory.
6. Does it remove $O(N^2)$ computation?	No for dense attention; it changes I/O behavior and avoids $O(N^2)$ HBM intermediates.	Exact pairwise scores remain.	Saying runtime becomes linear by definition.
7. How is causal masking optimized?	Skip fully invalid tiles and predicate invalid elements in partially valid diagonal tiles.	Coarse and fine masks.	Computing masked exponentials as normal values.
8. Why recompute in backward?	Recomputing tile scores is cheaper than storing/reading huge probability matrices on bandwidth-limited hardware.	Compute-memory tradeoff.	Assuming recomputation always wastes time.

Question	Clear answer	Expected key points	Common mistake
9. What limits tile size?	Shared memory, register capacity, warp mapping, occupancy, head dimension, and MMA layouts.	Resource tradeoffs.	Selecting the largest tile unconditionally.
10. When might it help less?	Very short sequences, unsupported shapes/types, launch-dominated cases, or workloads already dominated elsewhere.	Measure end-to-end.	Claiming uniform speedups.

7. Deep-Dive Questions

- Derive the numerator update.** If old $o = \text{sum_old} \exp(s - m_{\text{old}})v$, choose m_{new} ; rescale old o by $\exp(m_{\text{old}} - m_{\text{new}})$, then add $\text{sum_tile} \exp(s - m_{\text{new}})v$. Update l with the identical scaling so o/l is normalized correctly.
- What should forward save for backward?** Commonly output o and per-row log-sum-exp (or max/sum equivalent), plus RNG state for dropout. Backward can reconstruct probability tiles from Q/K and saved normalization.
- Why does loop order matter?** Keeping a Q tile resident while streaming K/V maximizes reuse of Q and retains one output accumulator. Alternative scheduling may improve K/V reuse across query tiles but increases synchronization or accumulator storage.
- How do dropout and reproducibility work without storing masks?** Use a counter-based RNG whose counter maps deterministically to logical attention elements, save seed/offset, and regenerate the same mask in backward.
- How does paged attention differ?** Paged attention addresses non-contiguous KV -cache blocks for inference. It may use online softmax tiling, but its central concern is decode-time cache layout and indirection rather than training's full score intermediates.

8. Comparison Tables

Property	Standard attention	FlashAttention-style
Dense arithmetic	$O(N^2d)$	$O(N^2d)$
Full score/probability in HBM	Yes	No
Kernel boundaries	GEMM, softmax, GEMM	Fused tiled kernel
Softmax	Whole row materialized	Online tile merge
Backward strategy	May save probabilities	Commonly recomputes tiles
Tiling decision	Larger tile benefit	Larger tile cost
Query tile	More K/V work amortization	Larger output accumulator
Key tile	Better MMA efficiency	More shared memory/scores
Pipeline stages	Better latency overlap	More shared memory and complexity

9. Common Mistakes

- Describing FlashAttention as sparse or approximate.
- Claiming it changes dense attention arithmetic from quadratic to linear.
- Normalizing each K tile independently.
- Rescaling the denominator but not the output numerator.
- Assuming fusion automatically wins for every short or irregular shape.
- Ignoring dropout RNG reproducibility in backward.

10. Edge Cases / Special Cases

- Entirely masked rows need a defined zero/NaN policy.
- Diagonal causal tiles mix valid and invalid lanes.
- Head dimensions not aligned to MMA/vector widths need masked fragments.
- Variable-length sequences need bounds per batch item.
- Extreme logits test online rescaling and FP32 statistics.
- Dropout, attention bias, ALiBi, and grouped-query layouts complicate fusion and indexing.

11. How to Explain in Interview

“FlashAttention computes the same dense attention but tiles it so scores and probabilities never go to HBM. A query tile stays on chip while key/value tiles stream through. Online softmax maintains a running max, denominator, and weighted-value numerator, rescaling old state when the maximum changes. The gain comes from lower memory traffic, not from skipping the quadratic dot products.”

12. Quick Revision Notes

- Exact dense attention; IO-efficient, not asymptotically fewer FLOPs.
- Running state: max m , exponential sum l , output numerator o .
- If max changes, rescale both l and o .
- Backward commonly recomputes score tiles.
- Trap: tile-local softmax is not global softmax.

13. Practice Tasks

1. Implement online softmax over scalar chunks and compare to full-row stable softmax.
2. Extend the state to a weighted vector numerator.
3. Draw HBM reads/writes for standard and tiled attention.
4. Implement a small CPU tiled attention reference with causal masking.
5. Estimate shared-memory/register needs for candidate tile sizes.
6. Benchmark a library FlashAttention path against materialized attention across sequence lengths.

14. Final Cheat Sheet

Item	Must remember
Core definition	Exact attention tiled to keep quadratic intermediates off HBM.
Why it matters	Greatly reduces attention memory traffic and activation storage.

Item	Must remember
Most asked	Online softmax, rescaling, IO complexity, backward recomputation, causal tiles.
Main comparison	Same dense math; different memory schedule.
One-line answer	"Stream K/V tiles past resident Q/output state and merge softmax online without materializing scores."

Quantization Kernels

1. Overview

Definition. Quantization represents tensors with fewer bits by mapping real values to integers or compact floating formats. A common affine integer mapping is:

```
q = clamp(round(x / scale) + zero_point, q_min, q_max)
x_hat = scale * (q - zero_point)
```

Quantization kernels compute scales, quantize/dequantize values, pack sub-byte data, or perform low-precision matrix multiplication with appropriate accumulation and rescaling. They matter because models are often limited by memory capacity/bandwidth and inference cost. Interviewers ask about quantization to test numeric representation, reductions, packing, Tensor Core/dot-product instructions, layout, calibration, and accuracy-performance tradeoffs.

2. Core Idea

Quantization is like storing temperatures to the nearest degree instead of with unlimited decimals. You save space, but introduce bounded rounding and clipping error. The scale chooses what one integer step means; the zero point lets integer zero represent real zero in asymmetric formats.

Suppose signed INT8 uses symmetric scale $s = \max(\text{abs}(x)) / 127$. For $x = [-1.0, 0.2, 0.9]$, $s \approx 0.007874$, yielding approximately $q = [-127, 25, 114]$. Dequantization gives values close to the originals.

A GPU quantization pipeline may:

1. Reduce a tensor/group to find range or absolute maximum.
2. Compute scale (and possibly zero point).
3. Multiply by inverse scale, round, clamp, and convert.
4. Pack INT4 values if needed.
5. In a consumer kernel, load compact values, unpack/dequantize or use integer MMA, accumulate in wider precision, and apply output scale.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Symmetric quantization	Zero point is usually zero; range centered around zero.	INT8 weights in <code>[-127, 127]</code> .	Simpler math, possibly wasted range for asymmetric data.
Asymmetric quantization	Scale plus nonzero zero point maps arbitrary min/max.	UINT8 activations.	Extra correction terms in GEMM.
Granularity	One scale per tensor, channel, row, or group.	INT4 weights with group size 128.	Accuracy versus metadata/work.
Static/dynamic	Scales come from calibration or are computed at runtime.	Dynamic activation quantization.	Reduction overhead versus adaptability.
Weight-only quantization	Weights are low-bit; activations remain FP16/BF16.	LLM INT4 inference.	Dequantize inside GEMM.
Packing	Multiple low-bit values share a byte/word.	Two INT4 nibbles per byte.	Signed decode and coalesced layout.
Accumulation	Products use wider type such as INT32 or FP32.	INT8 dot product accumulates into INT32.	Overflow bounds and rescaling.
Outliers	Rare large values can set a poor scale for most values.	Keep outlier channels higher precision.	Clipping/smoothing/mixed precision.

4. Real-World Example

In weight-only INT4 LLM inference, compressed weights halve memory again relative to INT8 and greatly reduce HBM traffic. The GEMM kernel loads packed weights, unpacks nibbles, applies per-group scales, multiplies them with FP16 activations, and accumulates in FP32. The useful speedup depends on whether reduced weight bandwidth outweighs unpack/dequantization overhead and whether the hardware has a suitable low-bit path.

5. Diagrams / Mental Models

```

floating tensor
  |
range/absmax reduction -> scale metadata
  |
scale -> round -> clamp -> integer values -> optional bit packing
                                   |
                                   low-bit GEMM/load <-+
                                   |

```

```

wide accumulation + rescale
|
output datatype

```

Granularity	Number of scales	Accuracy	Overhead
Per-tensor	1	Lowest flexibility	Lowest
Per-channel/row	One per channel/row	Better	Moderate
Per-group	One per small block	Often best	Highest metadata/dequant work

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is quantization?	Mapping values to a smaller representable set using scale and possibly zero point.	Rounding, clipping, dequantization.	Calling it lossless compression.
2. Symmetric versus asymmetric?	Symmetric centers integer range at zero; asymmetric shifts it with a zero point to fit non-centered ranges.	Simplicity versus range use.	Saying symmetric has no scale.
3. Why clamp after rounding?	Values outside the representable integer range would overflow/wrap or be invalid.	Saturation.	Casting directly and relying on implementation behavior.
4. Per-tensor versus per-channel?	Per-channel adapts to different ranges and usually improves accuracy but adds scale storage/indexing.	Granularity tradeoff.	Assuming finer is free.
5. Why use wider accumulators?	Dot products can exceed operand bit range quickly; INT8 products normally accumulate into INT32.	Overflow and precision.	Accumulating INT8 products in INT8.
6. Static versus dynamic quantization?	Static uses precomputed calibration scales; dynamic calculates scales from current inputs at runtime.	Runtime reductions/accuracy.	Confusing dynamic quantization with dynamic shapes.
7. What is weight-only quantization?	Only weights are stored low-bit; activations stay floating-point and weights are dequantized/fused during compute.	Bandwidth/capacity benefit.	Claiming arithmetic is necessarily pure integer.

Question	Clear answer	Expected key points	Common mistake
8. How is INT4 stored?	Usually two 4-bit codes per byte, with bit extraction and signed/offset interpretation.	Pack/unpack and layout.	Treating C++ <code>int4</code> as a standard scalar type.
9. Why do outliers hurt?	A few large magnitudes enlarge the scale step, reducing resolution for most values.	Clipping or finer groups.	Assuming max calibration is always optimal.
10. When does quantization speed up inference?	When lower memory traffic/capacity and supported low-bit compute outweigh scale, packing, conversion, and accuracy costs.	End-to-end measurement.	Equating 4x fewer bits with 4x speedup.

7. Deep-Dive Questions

- Derive asymmetric GEMM correction.** With real approximations $A = s_a(q_a - z_a)$ and $B = s_b(q_b - z_b)$, each dot product expands to $s_a s_b [\sum q_a q_b - z_b \sum q_a - z_a \sum q_b + K z_a z_b]$. Symmetric zero points remove correction terms.
- Can INT32 accumulation overflow?** Yes. A worst-case signed INT8 dot product is roughly $K * 127 * 127$; very large K , zero-point corrections, or accumulated partials need bounds analysis, split accumulation, or wider handling.
- How do you quantize efficiently per row?** First reduce row absmax/minmax, derive scale, then quantize. A fused single-pass method may buffer values on chip for small rows; otherwise two reads or a producer fusion is needed.
- What is the challenge in INT4 GEMM layout?** Packing must match how warp MMA/dequant instructions consume operands. A compact but poorly arranged format can spend more time permuting/unpacking than it saves in bandwidth.
- How do stochastic rounding and deterministic rounding differ?** Nearest rounding is deterministic and biased in some repeated-update settings; stochastic rounding chooses neighboring values probabilistically to preserve expectation, requiring efficient reproducible RNG.

8. Comparison Tables

Format	Bits/value	Typical accumulation	Strength	Limitation
FP32	32	FP32	Accuracy/range	Bandwidth/capacity
FP16	16	FP32 often	Fast Tensor Core support	Limited exponent range
BF16	16	FP32 often	FP32-like exponent range	Coarse mantissa
INT8	8	INT32/FP32	Mature inference support	Scale/calibration needed

Format	Bits/value	Typical accumulation	Strength	Limitation
INT4	4	INT32/FP32	Very compact weights	Packing and accuracy complexity

Scheme	Zero point	Best fit	Kernel implication
Symmetric	0	Roughly centered weights	Simple multiply/rescale
Asymmetric	Nonzero	Skewed/non-negative activations	Correction terms or pre-adjustment

9. Common Mistakes

- Converting without explicit rounding and saturation.
- Computing scale from `max` instead of absolute maximum for symmetric signed data.
- Forgetting zero-point correction in integer GEMM.
- Treating per-group metadata as free.
- Unpacking low-bit values into a full temporary tensor before GEMM.
- Assuming reduced model size guarantees equal accuracy or proportional speedup.

10. Edge Cases / Special Cases

- An all-zero group needs a nonzero fallback scale to avoid division by zero.
- NaN/infinity during calibration needs an explicit policy.
- Signed INT8 often uses `[-127, 127]` for symmetry although `-128` exists.
- Rounding ties and negative shifts must match the reference convention.
- Odd INT4 element counts require padding or nibble-tail handling.
- Scale underflow/overflow matters if metadata itself uses low precision.
- Quantized checkpoints are format- and layout-specific, not just raw integer arrays.

11. How to Explain in Interview

“Quantization maps floating values to low-bit codes using a scale and optionally a zero point, with explicit rounding and saturation. GPU kernels either quantize via a range reduction or consume packed values inside a fused low-bit GEMM. The real tradeoff is reduced memory traffic and capacity versus calibration error, scale metadata, unpacking, correction terms, and accumulator safety.”

12. Quick Revision Notes

- Affine mapping: `q=clamp(round(x/s)+z)`; dequantize with `s(q-z)`.
- Symmetric is simpler; asymmetric uses range more flexibly.
- Finer scale granularity improves accuracy but costs metadata/work.
- Accumulate low-bit products in a wider type.
- Trap: lower bit width does not guarantee proportional speedup.

13. Practice Tasks

1. Implement symmetric INT8 quantize/dequantize and measure error.
2. Add asymmetric min/max quantization and zero-point handling.

- 3. Pack and unpack signed INT4 values, including odd tails.
- 4. Compare per-tensor, per-row, and per-group error on an outlier-heavy matrix.
- 5. Derive accumulator overflow limits for given **K** and ranges.
- 6. Fuse dequantization into a small matrix multiplication and compare traffic with a full temporary.

14. Final Cheat Sheet

Item	Must remember
Core definition	Map real values to low-bit codes through scale, rounding, and saturation.
Why it matters	Reduces model storage and memory bandwidth; may unlock low-bit hardware.
Most asked	Symmetric/asymmetric, granularity, packing, accumulators, outliers, fusion.
Main comparison	Per-tensor is cheap; per-group/channel is more accurate but carries overhead.
One-line answer	"Reduce for scales, quantize with explicit rounding/clamping, and consume packed values with wide accumulation."

Final Cross-Kernel Interview Map

Kernel	Dominant GPU pattern	Main numerical issue	Main optimization
Softmax	Row reductions	Exponential overflow	Stable/online reductions and fusion
LayerNorm	Row statistics + elementwise	Variance cancellation	FP32/Welford and residual fusion
RMSNorm	Sum-of-squares reduction	Square/accumulation error	FP32 reduction and fusion
ReLU/GELU	Elementwise map	Approximation/subgradient	Vectorization and epilogue fusion
Cross entropy	Row reduction + target gather	$\log(0)$ /overflow	Fused log-sum-exp
Matrix multiplication	Tiled reduction	Accumulation error	Shared/register tiling and Tensor Cores
Attention	GEMM + softmax + GEMM	Masked stable normalization	Layout, fusion, KV-cache efficiency
FlashAttention-style	Tiled fused attention	Online rescaling	Keep quadratic intermediates off HBM
Quantization	Reduction + conversion/packing	Clipping/rounding/overflow	Fused low-bit consumption

Universal Interview Checklist

When asked to design any ML GPU kernel, state:

1. **Tensor shapes, axis, layout, and datatype.** Most indexing mistakes begin here.
2. **Thread ownership.** Say which warp/block computes which row, tile, or output.
3. **Memory movement.** Identify global reads/writes, reuse, coalescing, and avoidable intermediates.
4. **Cooperation.** Explain reductions, synchronization, and neutral values for invalid lanes.
5. **Numerics.** Name accumulator precision, stabilization, rounding, and validation tolerance.
6. **Edges.** Cover masks, tails, non-contiguous strides, empty sizes, and non-finite inputs.
7. **Performance proof.** Benchmark representative shapes, warm up, synchronize correctly, and compare with an optimized library/reference.